Automated CSV Processing and Google Drive File Transfer System

---

1. Project Title

Automated CSV Processing and Google Drive File Transfer System using Python and Google Drive API

---

2. Project Overview

The Automated CSV Processing and Google Drive File Transfer System is a Python-based automation solution designed to process CSV files stored in Google Drive. The system automatically identifies the latest CSV file from a specified Google Drive folder, downloads the file, splits the data into multiple individual CSV files based on a unique identifier, uploads the generated files to an output folder, and transfers the processed files to a client's Google Drive folder.

This automation reduces manual file processing efforts, improves data handling efficiency, and provides a scalable solution for managing large CSV datasets.

---

3. Problem Statement

In many organizations, large CSV files containing customer or transactional data need to be separated into multiple individual files before sharing them with different teams or clients.

The traditional manual process involves:

- Downloading CSV files manually.
- Identifying required records.
- Creating multiple individual CSV files.
- Uploading files one by one.
- Sharing files with clients.

This process is time-consuming, error-prone, and inefficient when handling large datasets.

To overcome these challenges, an automated CSV processing pipeline was developed.

---

4. Objective

The main objectives of this project are:

- To automate CSV file processing using Python.
- To retrieve files automatically from Google Drive.
- To identify the latest uploaded CSV file from a folder.
- To split CSV data into separate files based on unique IDs.
- To automatically upload generated files to Google Drive.
- To transfer processed files to client Google Drive folders.
- To reduce manual intervention and improve productivity.

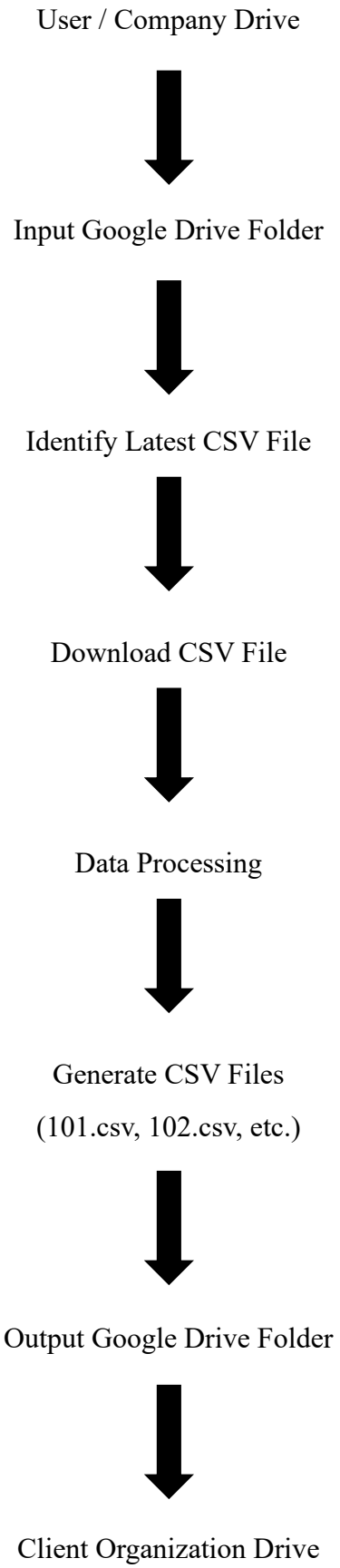
---

5. Technologies Used

Technology	**Purpose**
Python	Core programming language
Google Colab	Development and execution environment
Pandas	CSV data processing
Google Drive API	Drive file operations
Google Authentication	Secure access management
Google Workspace Drive	File storage and sharing

---

6. System Architecture



7. System Workflow

Step 1: Google Drive Authentication

The system authenticates the user using Google Drive API.

Authentication provides secure access to:

- Read files from Google Drive.
- Download CSV files.
- Upload processed files.
- Transfer files between folders.

The authentication process eliminates manual file handling and enables automated operations.

Step 2: Input Folder Identification

The system uses a Google Drive folder ID as input.

Example:

INPUT_FOLDER_ID

The application searches inside the specified folder and identifies available CSV files.

Step 3: Latest CSV File Detection

Instead of manually providing a file ID, the system automatically finds the latest CSV file.

The process:

1. Access the input folder.
2. Filter only CSV files.
3. Sort files based on modification time.
4. Select the latest uploaded CSV file.

Benefits:

- No manual file ID update required.
- Supports changing input files.
- Suitable for continuous automation.

Step 4: CSV File Download

The selected CSV file is downloaded from Google Drive into the execution environment.

The downloaded file is temporarily stored for processing.

Example:

downloads/

customer_data.csv

---

8. CSV Splitting Process

The downloaded CSV file is processed using Pandas.

The system identifies a unique column from the dataset.

Supported ID columns:

cust_id

customer_id

id

ID

CustID

CustomerID

customerId

If predefined columns are unavailable, the system automatically searches for a unique column.

Splitting Logic

Example Input:

cust_id	name	amount
101	John	500
102	David	700
103	Sam	900

Generated Output:

101.csv

102.csv

103.csv

Each generated file contains only one row of original data.

---

9. Output File Upload

After splitting, the generated CSV files are uploaded automatically to the configured output folder.

Example:

Output Folder

|

├──101.csv

├──102.csv

├──103.csv

The upload process eliminates manual uploading.

---

10. Client File Transfer

The final processed files are transferred to the client's Google Drive folder.

The transfer process:

1. Reads files from output folder.
2. Copies files to client folder.
3. Maintains original file names.
4. Completes automatic data delivery.

The client only needs proper folder access permission.

---

11. Error Handling

The system handles common issues:

Authentication Errors

Solution:

- Verify Google account authorization.
- Refresh authentication token.

Permission Errors

Example:

403 Insufficient Permission

Solution:

- Provide Editor access to required Google Drive folders.

Missing CSV Files

Solution:

- Check input folder.
- Ensure CSV file exists.

---

12. Advantages

- Fully automated CSV processing.
- Reduces manual effort.
- Faster data distribution.
- Minimizes human errors.
- Supports large datasets.
- Easy integration with Google Drive.
- Scalable for business workflows.

---

13. Future Enhancements

The system can be improved with:

1. Web Interface

Develop a Flask-based UI where users can enter:

- Input Folder ID
- Output Folder ID
- Client Folder ID

---

2. Scheduling Automation

Run the process automatically:

- Daily
- Weekly
- Monthly

using cloud scheduling services.

---

3. Email Notification

Send automatic notifications after successful processing.

Example:

"CSV processing completed successfully."

---

4. Database Integration

Store:

- Processed file details.
- Processing history.
- Error logs.

---

5. Cloud Deployment

Deploy the application using:

- Google Cloud Run
- Google Compute Engine

for production usage.

14. Conclusion

The Automated CSV Processing and Google Drive File Transfer System successfully automates the complete workflow of CSV file handling. The system eliminates repetitive manual tasks by automatically detecting, processing, splitting, uploading, and transferring CSV files between Google Drive locations.

This project demonstrates practical implementation of Python automation, data processing, cloud API integration, and file management techniques suitable for real-world business applications.